std::hive and containers like it are not a good fit for the standard library

Document #:	P3001R0
Date:	2023-10-15
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Jonathan Müller <foonathan@jonathanmueller.dev> Zach Laine <whatwasthataddress@gmail.com> Bryce Adelstein Lelbach <brycelelbach@gmail.com> David Sankel <dsankel@adobe.com>

Contents

§ 1 Abstract

The high-performance `std::hive` container is proposed for inclusion in the C++ standard. While the reference implementation is useful in many contexts, it is yet unclear whether standardization of its interface it appropriate. This paper attempts to answer this question

by capturing the characteristics of successful standardized libraries and considering the unique requirements of high-performance containers. We conclude that evolutionary limitations and high standardization costs make standardization of libraries such as `std::hive` undesirable.

§ 2 Introduction

At the Varna meeting, the authors raised concerns about the appropriateness of [P0447R22]'s `std::hive` as an addition to the standard library. Let's look at why we are concerned. First, let's discuss what should and should not go in the standard library. Then we will argue that `std::hive` is not a good fit.

§ 3 What the C++ standard library is good at

Elements of the standard library ideally fall into one of the following categories [stdlib-bryce,stdlib-corentin,stdlib-jonathan,stdlib-titus]:

§ 3.1 Types and functions requiring compiler intrinsics

The standard library is the only place where we can put types and functions that require compiler support, since it is shipped by and often developed alongside a C++ compiler implementation. This includes things like `std::initializer_list`, some `<type_traits>`, or `std::coroutine_traits`.

§ 3.2 Core vocabulary types

C++ libraries and applications want to use user-defined types like `optional`, `span`, or `string_view` to communicate intent and provide more expressive APIs.

Consider `optional`. If every library shipped with its own implementation, communication between them would require programmer and CPU time to translate between types. Putting an `optional` implementation into the standard library alleviates that problem, since all libraries can use the standard library.

§ 3.3 Cross-platform OS abstractions

The standard library is ubiquitous and implemented by platform experts. Most platforms provide I/O, threading, and memory allocation. If this common OS subset is standardized, vendors can implement it for their platforms with their expertise, and users everywhere can rely on a simple, portable interface.

§ 3.4 Fundamental algorithms and data structures

Some types (e.g. dynamically-allocated arrays, stacks, and queues), and algorithms (e.g. sorting and searching), are fundamental to most or all programming tasks. Working in C++ without `vector` or `sort` would be significantly more painful than working in C++ today. The types and algorithms in this category are needed with high enough frequency that we would not want users to have to write them. They also have widely- and easily-understood semantics, and *well-established*, *stable* implementations.

They are distinct from vocabulary types, in that vocabulary types are important for establishing conventions, whereas the entities here are important for getting work done without reinventing those entities – regardless of whether they are used to interoperate with other code.

§ 4 Limitations of standardized libraries

For better and worse, the C++ standard library maintains a stable ABI and API: Deviations cause significant user disruption. Proposal authors need to be aware that as soon as something is standardized, it is essentially done. The committee has decided against a "standard library 2.0", so whatever facility was standardized, we have to live with it.

Yes, the committee has changed the ABI of `std::string`, deprecated and removed egregiously wrong facilities, and recently approved a significant number of DRs against the C++20 standard library. However, these kind of changes are exceptional. Facilities that are bad but insufficiently terrible like `std::vector<bool>`, `std::unordered_map`, or `std::regex` are going to stick around.

The committee thus cannot standardize facilities without an established interface: Once standardized, a library's API and ABI is effectively frozen, unlike non-standard libraries which can continue to evolve. To a lesser extent, the same is also true for its implementation.

Standardizing a feature takes a lot of work, and the committee has limited time. Everything we discuss takes time away from a different feature and means delaying something else. The committee thus need to be absolutely sure we want a huge feature, like graphics or networking, before investing significant time.

A standardized proposal needs to be portable across all platforms and will have multiple competing implementations of varying quality. The committee thus needs to be careful standardizing APIs that are not available on all platforms or where users want to rely on certain implementation characteristics such as its performance.

§ 5 High-performance containers and the C++ standard library

A high-performance container is a container implementation that is used specifically for its runtime behavior or memory usage. Examples are the Abseil or Boost hash tables,

LLVM's small vector implementation, or the proposed `std::hive`. Such containers have the following qualities:

— A Big O complexity better than other implementations.

— A runtime or memory usage that is measurably better than other implementations in the relevant micro and macro benchmarks: Switching to a high-performance container is decided only after benchmarking alternative implementations.

— Need not be a vocabulary type: The cost of using a custom type for a specific part of the API is worth it for performance.

— Is actively maintained. CPUs continue changing, and better algorithms are available all the time. High-performance container implementations must adapt to those changes and keep being improved. Otherwise, users will switch to a more modern, faster implementation.

These qualities are at odds with standardized C++ library facilities.

Since high performance containers do not require compiler support or OS APIs and are not a vocabulary types, they miss out on the core benefits of being in the standard library. Instead, such libraries would inherit only the downsides:

— Stability requirements seriously impede evolution. Simple changes like adding additional data members are ABI breaks, and most of the internal implementation is exposed in API guarantees like element stability.

— Standardization requires a significant amount of committee time that could be spent on more appropriate additions.

— We only standardize an interface, not an implementation: It will be implemented by multiple vendors. Even if one implementation is good, performance can vary between platforms. You do not have a performance guarantee when using a standard library feature.

At best, standardizing a high-performance container means it is available without relying on external libraries. At worst, standardizing a high-performance container takes months of committee time, ends up with something that is already obsolete by the time it is finally standardized, and cannot be updated due to ABI concerns.

How many `std:unordered_maps` or `std::regexes` do we want in the standard library?

§ 6 `std::hive` and the C++ standard library

[P0447R22]'s `std::hive` is a high-performance container, so all of the above points apply. It is undeniably a useful container, and the provided reference implementation seems solid. We have use-cases for it in our own projects.

However, **we are not going to standardize the reference implementation, we are going to standardize an interface.**

The interface leaves enough room to the standard library implementers to make their own trade-offs, while at the same time being specific enough that later optimizations might be breaking changes. We cannot imagine a scenario where we care enough about performance to use something like `std::hive<T>` over a `std::vector<std::unique_ptr<T>>` (maybe paired with a hash map to have efficient access from `T*` to index), but do not care enough about performance that we are just fine with whatever the quality of the standard library implementation is—as opposed to the guarantee from a specific external library.

Even if we ignore the downsides of standardizing a high-performance container, what are the upsides?

It does not rely on compiler magic or OS APIs, so it does not need to be in the standard library. Is it a vocabulary type? It used to have a "priority" policy parameter and still has an allocator. Types with user customizable policies are not usually vocabulary types since different libraries might pick different policies, making them incompatible. Is it fundamental to many programming tasks—that is, is it so frequently needed that end users frequently need to invent it? While the author argues that it is frequently needed in his domain, the reference implementation uses novel algorithms. It is not a `std::vector` or `std::find` that would be implemented the same everywhere if not in the standard. It also seems like it is an area of active implementation improvements, which is not possible with standardized containers.

That leaves convenience. Adding it to the standard library makes it easier to use by others since it does not require setting up a build system, package manager, or some other mechanism to get third-party libraries. But is it going to be used by projects that do not already have third-party dependencies? If not, the cost of adding yet another third-party library is negligible.

So if we do not have any guarantee that the final implementation is performant enough, and there is not a clear upside to standardizing it, why should we take time out of the C++26 cycle on wording review of `std::hive` in favor of SIMD, Unicode, or executors?

§ 7 References

[P0447R22] Matt Bentley. 2023-05-17. Introduction of std::hive to the standard library. https://wg21.link/p0447r22

[stdlib-bryce] Bryce Adelstein Lelbach. What Belongs In The C++ Standard Library? https://www.youtube.com/watch?v=OgM0MYb4DqE

[stdlib-corentin] Corentin Jabot. A cake for your cherry: what should go in the C++ standard library? https://hackernoon.com/a-cake-for-your-cherry-what-should-go-in-the-c-standard-library-804fcecccef8

[stdlib-jonathan] Jonathan Müller. What should be part of the C++ standard library? https://www.foonathan.net/2017/11/standard-library/

[stdlib-titus] Titus Winters. What Should Go Into the C++ Standard Library. https://abseil.io/blog/20180227-what-should-go-stdlib